

宁波大学
NINGBO UNIVERSITY

宁波大学信息学院

实验报告

实验课程名称： 嵌入式系统

学　　院： 信息科学与工程学院

专　　业： 计算机科学与技术

指 导 教 师： 王翀,王艺睿

报告人姓名： 顾凯琪 **学号：** 226001624

班　　级： 2022 级计算机 2 班

学　　期： 2024-2025 学年第二学期

信息学院实验中心制

实验								
成绩								

实验项目名称	**实验九 嵌入式文件 I/O 编译实验**

一、实验目的与要求：

1.1 实验目的

(1) 理解 Linux 系统中文件 I/O 操作的基本概念和原理。

(2) 学习使用标准 I/O 库函数进行文件的读写操作。

(3) 掌握文件描述符的使用，以及如何在用户空间通过系统调用来访问和操作文件。

1.2 实验要求

(1) 熟悉 Linux 系统中文件的分类，包括普通文件、目录文件、链接文件和设备文件。

(2) 编写程序实现对文件的打开、读写等基本操作，体会文件描述符在文件 I/O 中的作用。

(3) 通过实验加深对虚拟文件系统的理解，了解其在 Linux 系统中的工作原理和作用。

(4) 掌握基于文件描述符的 I/O 操作，包括但不限于 read、write、lseek 等系统调用的使用。

二、实验设备：

(1) 华清远见开发环境。

(2) 实验平台为华清远见提供的虚拟机环境。

(3) 使用的操作系统是 Ubuntu 12.04 LTS 64-bit。

(4) 虚拟机工具软件为 VMware Player。

三、实验方法（原理、流程图）

实验原理：

1. 系统调用

所谓系统调用是指操作系统提供给用户程序调用的一组“特殊”接口，用户程序可以通过这组“特殊”接口获得操作系统内核提供的服务。例如用户可以通过进程控制相关的系统调用来创建进程、实现进程之间的通信等。

在这里，为什么用户程序不能直接访问系统内核提供的服务呢？这是由于在 Linux 中，为了更好地保护内核空间，将程序的运行空间分为内核空间和用户空间（也就是常称的内核态和用户态），它们分别运行在不同的级别上，逻辑上是相互隔离的。因此，用户进程在通常情况下不允许访问内核数据，也无法使用内核函数，它们只能在用户空间操作用户数据，调用用户空间的函数。

但是，在有些情况下，用户空间的进程需要获得一定的系统服务（调用内核空间程序），这时操作系统就必须利用系统提供给用户的“特殊接口”——系统调用规定用户进程进入内核空间的具体位置。进行系统调用时，程序运行空间需要从用户空间进入内核空间，处理完后再返回到用户空间。

Linux 系统调用非常精简（只有 250 个左右），它继承了 Unix 系统调用中最基本和最有用的部分。这些系统调用按照功能逻辑大致可分为进程控制、进程间通信、文件系统控制、存储管理、网络管理、套接字控制、用户管理等几类。

2. 用户编程接口（API）

前面讲到的系统调用并不直接与程序员进行交互，它仅仅是一个通过软中断机制向内核提交请求以获取内核服务的接口。实际使用中程序员调用的通常是用户编程接口——API。

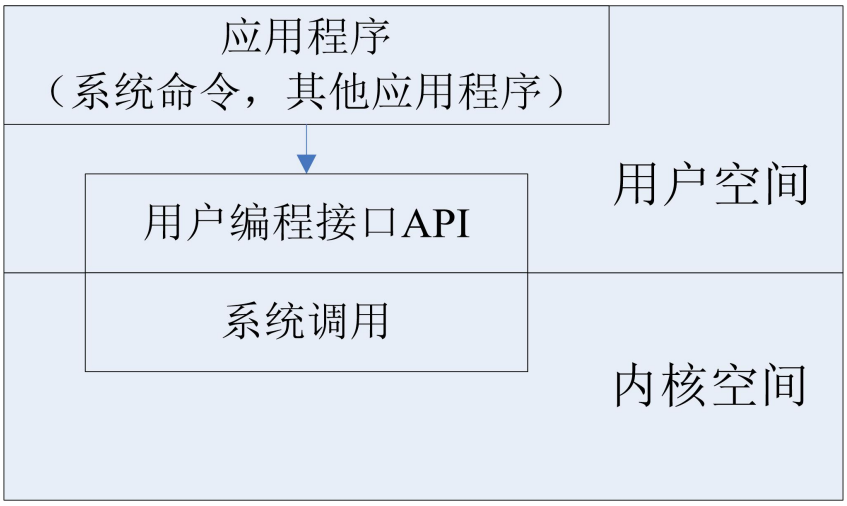
例如创建进程的 API 函数 frok()对应于内核空间的 sys_fork()系统调用。但并不是所有的函数都对应一个系统调用，有时，一个 API 函数会需要几个系统调用来共同完成函数的功能，

甚至还有一些 API 函数不需要调用相应的系统调用（因此它所完成的不是内核提供的服务）。

在 Linux 中，用户编程接口（API）遵循了在 Unix 中最流行的应用编程界面标准——POSIX 标准。POSIX 标准是由 IEEE 和 ISO/IEC 共同开发的标准系统。该标准基于当时现有的 Unix 实践和经验，描述了操作系统的系统调用编程接口（实际上就是 API），用于保证应用程序可以在源代码一级上在多种操作系统上移植运行。这些系统调用编程接口主要是通过 C 库（libc）实现的。

3. 系统命令

系统命令相对 API 更高了一层，它实际上一个可执行程序，它的内部引用了用户编程接口（API）来实现相应的功能。它们之间的关系如下图所示。

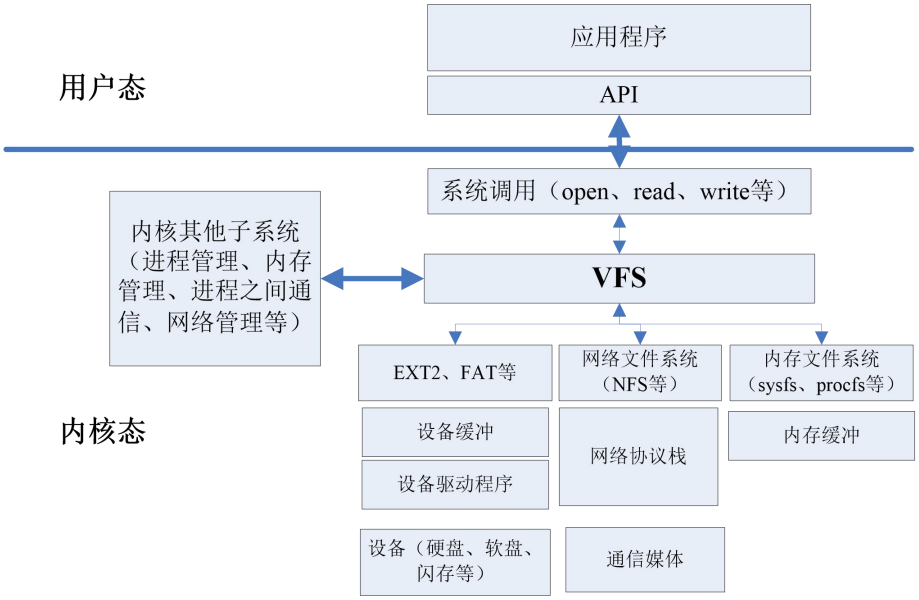


4. 虚拟文件系统（VFS）

Linux 系统成功的关键因素之一就是具有与其他操作系统和谐共存的能力。Linux 的文件系统由两层结构构建。第一层是虚拟文件系统（VFS），第二层是各种不同的具体的文件系统。

VFS 就是把各种具体的文件系统的公共部分抽取出来，形成一个抽象层，是系统内核的一部分。它位于用户程序和具体的文件系统之间。它对用户程序提供了标准的文件系统调用接口，对具体的文件系统（比如：Ext2，FAT32 等），它通过一系列的对不同文件系统公用的函数指针来实际调用具体的文件系统函数，完成实际的各有差异的操作。任何使用文件系统的程序必须经过这层接口来使用它。通过这样的方式，VFS 就对用户屏蔽了底层文件系统的实现细节和差异。

VFS 不仅可以对具体文件系统的数据结构进行抽象，以一种统一的数据结构进行管理，并且还可以接受用户层的系统调用，例如：open()、read()、write()、stat()、link()等。此外，它还支持多种具体文件系统之间的相互访问，接受内核其他子系统的操作请求，例如内存管理和进程调度。VFS 在 Linux 系统中的位置如下图所示。



通过以下命令可以查看系统中支持哪些文件系统：

```
$ cat /proc/filesystems
nodev    sysfs
nodev    rootfs
……
nodev    tmpfs
nodev    pipefs
……
           ext2
nodev    ramfs
nodev    hugetlbfs
           iso9660
nodev    mqueue
nodev    selinuxfs
           ext3
nodev    rpc_pipefs
……
```

5. Linux 中文件及文件描述符

Linux 操作系统都是基于文件概念的。文件是以字符序列而构成的信息载体。根据这一点，可以把 I/O 设备当作文件来处理。因此，与磁盘上的普通文件进行交互所用的同一系统调用可以直接用于 I/O 设备。这样大大简化了系统对不同设备的处理，提高了效率。Linux 中的文件主要分为 4 种：普通文件、目录文件、链接文件和设备文件。

那么，内核如何区分和引用特定的文件呢？这里用到了一个重要的概念——文件描述符。对于 Linux 而言，所有对设备和文件的操作都是使用文件描述符来进行的。文件描述符是一个非负的整数，它是一个索引值，并指向在内核中每个进程打开文件的记录表。当打开一个现存文件或创建一个新文件时，内核就向进程返回一个文件描述符；当需要读写文件时，也需要把文件描述符作为参数传递给相应的函数。

通常，一个进程启动时，都会打开 3 个文件：标准输入、标准输出和标准出错处理。这

3 个文件分别对应文件描述符为 0、1 和 2（也就是宏替换 STDIN_FILENO、STDOUT_FILENO 和 STDERR_FILENO，鼓励读者使用这些宏替换）。

基于文件描述符的 I/O 操作虽然不能直接移植到类 Linux 以外的系统上去（如 Windows），但它往往是实现某些 I/O 操作的惟一途径，如 Linux 中低层文件操作函数、多路 I/O、TCP/IP 套接字编程接口等。同时，它们也很好地兼容 POSIX 标准，因此，可以很方便地移植到任何 POSIX 平台上。基于文件描述符的 I/O 操作是 Linux 中最常用的操作之一，希望读者能够很好地掌握。

6. 标准 I/O 编程

本章前面几节所述的文件及 I/O 读写都是基于文件描述符的。这些都是基本的 I/O 控制，是不带缓存的。而本节所要讨论的 I/O 操作都是基于流缓冲的，它是符合 ANSI C 的标准 I/O 处理，这里有很多函数读者已经非常熟悉了（如 printf()、scantf()函数等），因此本节中仅简要介绍最主要的函数。

前面讲述的系统调用是操作系统直接提供的函数接口。因为运行系统调用时，Linux 必须从用户态切换到内核态，执行相应的请求，然后再返回到用户态，所以应该尽量减少系统调用的次数，从而提高程序的效率。

标准 I/O 提供流缓冲的目的是尽可能减少使用 read()和 write()等系统调用的数量。标准 I/O 提供了 3 种类型的缓冲存储。

- 全缓冲：在这种情况下，当填满标准 I/O 缓存后才进行实际 I/O 操作。对于存放在磁盘上的文件通常是由标准 I/O 库实施全缓冲的。标准 I/O 尽量多读写文件到缓冲区，当缓冲区已满或手动 flush 时才会进行磁盘操作。
- 行缓冲：在这种情况下，当在输入和输出中遇到行结束符时，标准 I/O 库执行 I/O 操作。这允许我们一次输出一个字符（如 fputc()函数），但只有写了一行之后才进行实际 I/O 操作。标准输入和标准输出就是使用行缓冲的典型例子。
- 不带缓冲：标准 I/O 库不对字符进行缓冲。如果用标准 I/O 函数写若干字符到不带缓冲的流中，则相当于用系统调用 write()函数将这些字符全写到被打开的文件上。标准出错 stderr 通常是不带缓存的，这就使得出错信息可以尽快显示出来，而不管它们是否含有一个行结束符。

在下面讨论具体函数时，请读者注意区分以上的三种不同情况。

1．打开文件

打开文件有三个标准函数，分别为：fopen()、fdopen()和 freopen()。它们可以以不同的模式打开，但都返回一个指向 FILE 的指针，该指针指向对应的 I/O 流。此后，对文件的读写都是通过这个 FILE 指针来进行。其中 fopen()可以指定打开文件的路径和模式，fdopen()可以指定打开的文件描述符和模式，而 freopen()除可指定打开的文件、模式外，还可指定特定的 I/O 流。

其中，mode 类似于 open()函数中的 flag，可以定义打开文件的访问权限等，表 4.17 说明了 fopen()中 mode 的各种取值。

2．关闭文件

关闭标准流文件的函数为 fclose()，该函数将缓冲区内的数据全部写入到文件中，并释放系统所提供的文件资源。

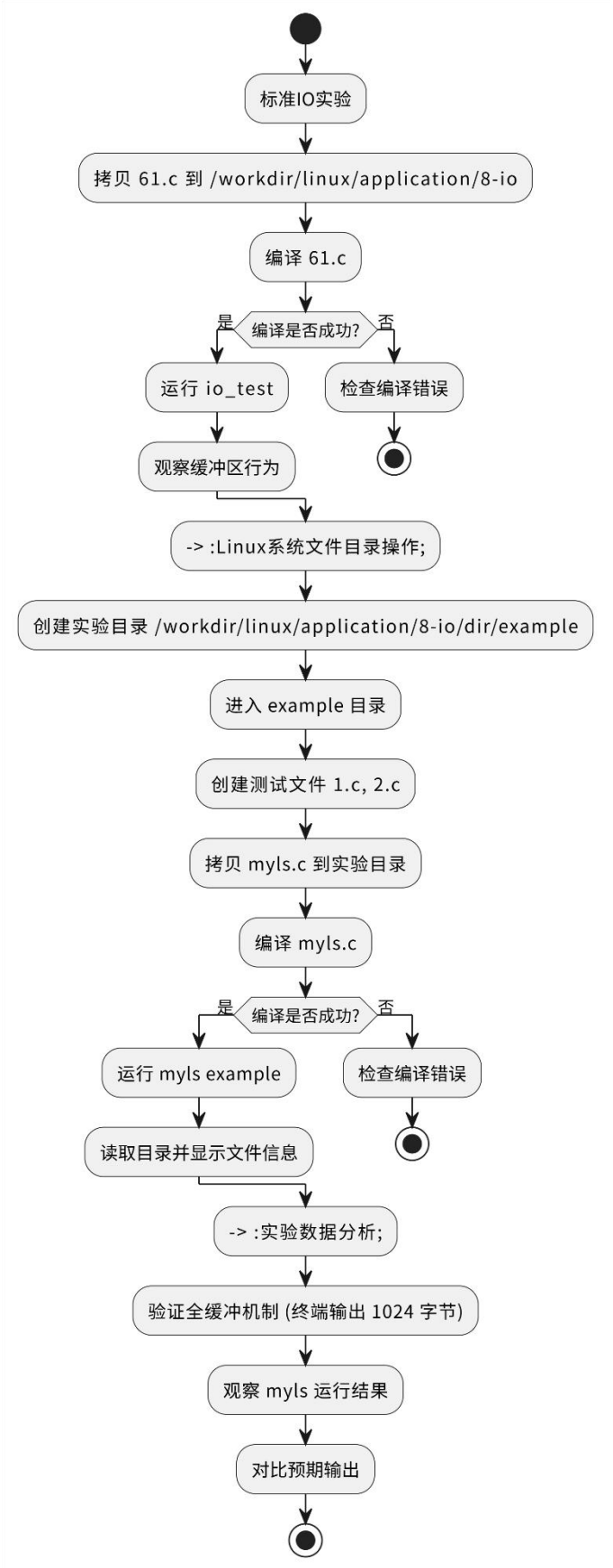
3．读文件

在文件流被打开之后，可对文件流进行读写等操作，其中读操作的函数为 fread()。

4．写文件

fwrite()函数是用于对指定的文件流进行写操作。

流程图：


标准IO实验
拷贝 61.c 到 /workdir/linux/application/8-io
编译 61.c
编译是否成功?
是
否
运行 io_test
检查编译错误
观察缓冲区行为
-> :Linux系统文件目录操作;
创建实验目录 /workdir/linux/application/8-io/dir/example
进入 example 目录
创建测试文件 1.c, 2.c
拷贝 myls.c 到实验目录
编译 myls.c
编译是否成功?
是
否
运行 myls example
检查编译错误
读取目录并显示文件信息
-> :实验数据分析;
验证全缓冲机制 (终端输出 1024 字节)
观察 myls 运行结果
对比预期输出

四、实验过程、步骤及内容

4.1 标准 IO 实验

本实验通过一个简单的程序计算默认缓冲区的大小，理解标准 I/O 提供的三种类型的缓存，这三种缓存类型分别是：

全缓冲：当填满 I/O 缓存后才进行实际的 I/O 操作；

行缓冲：当在输入和输出中遇到新换行符(‘\n’)时，进行 I/O 操作；

不带缓冲：标准 I/O 库不对字符进行缓冲，例如 stderr；

本实验验证全缓存类型。

将“华清远见-CORTEXA9 资料： \实验代码\2、Linux 移植驱动及应用\1、Linux 应用\实验代码\06. Linux 系统标准 IO 实验\实验代码”目录下的“61.c”拷贝到【~/workdir/linux/application/8-io】目录下并编译：

```
$ gcc 61.c -o io_test
$ ./io_test
```

```
linux@ubuntu64-vm:~/workdir/linux/application$ cp /mnt/hgfs/share/实验代码1/06.\
 Linux系统标准IO实验/实验代码/* 8-io/ -a
linux@ubuntu64-vm:~/workdir/linux/application$ ls
5-shell  6-gcc  8-io
linux@ubuntu64-vm:~/workdir/linux/application$ cd 8-io
linux@ubuntu64-vm:~/workdir/linux/application/8-io$ ls
61.c  04  dir
linux@ubuntu64-vm:~/workdir/linux/application/8-io$ gcc 61.c -o io_test
linux@ubuntu64-vm:~/workdir/linux/application/8-io$ ./io_test
000001002003004005006007008009010011012013014015016017018019020021022023024025026
027028029030031032033034035036037038039040041042043044045046047048049050051052053
054055056057058059060061062063064065066067068069070071072073074075076077078079080
081082083084085086087088089090091092093094095096097098099100101102103104105106107108
109110111112113114115116117118119120121122123124125126127128129130131132133134135
136137138139140141142143144145146147148149150151152153154155156157158159160161162
163164165166167168169170171172173174175176177178179180181182183184185186187188189
190191192193194195196197198199200201202203204205206207208209210211212213214215216
217218219220221222223224225226227228229230231232233234235236237238239240241242243
244245246247248249250251252253254255256257258259260261262263264265266267268269270
271272273274275276277278279280281282283284285286287288289290291292293294295296297
298299300301302303304305306307308309310311312313314315316317318319320321322323324
325326327328329330331332333334335336337338339340341342343344345346347348349350351
352353354355356357358359360361362363364365366367368369370371372373374375376377378
379380381382383384385386387388389390391392393394395396397398399400401402403
```

实验代码

```
#include <stdio.h>
int main()
{
    int i=0;
    for(i=0;i<379;i++)//每次向缓冲区内写三个字符
    {
        if(i>=100)
            fprintf(stdout,"%d",i);
        else if (i>=10)
            fprintf(stdout,"0%d",i);
        else if (i>=0)
            fprintf(stdout,"00%d",i);
    }
    while(1);//强制执行，如果取消，程序结束时将会输出所有字符，看不到效果了。
}
```

4.2 Linux 系统文件目录操作编程实验

运行 ubuntu 系统，打开命令行终端。

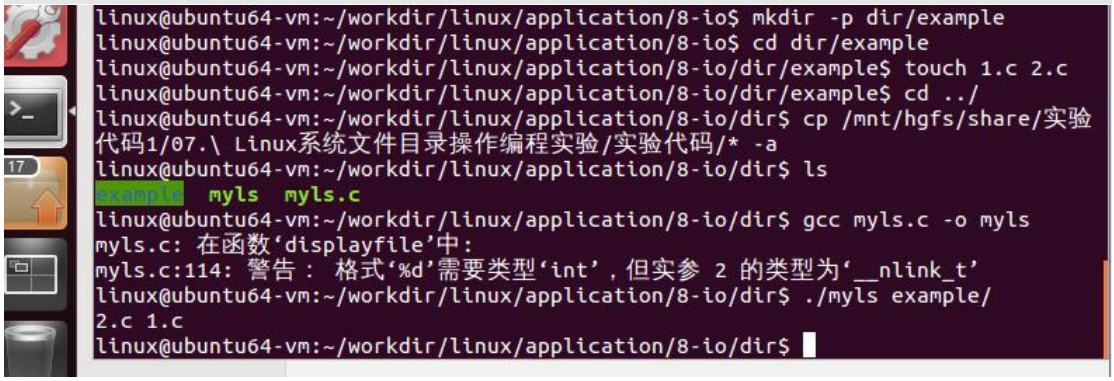
执行命令,创建实验目录：

```
$ cd ~/workdir/linux/application/8-io
$ mkdir -p dir/example
$ cd dir/example
$ touch 1.c  2.c    //在 example 目录下创建两个文件，后面程序会读这两个文件
$ cd ../
```

将“华清远见-CORTEXA9 资料： \实验代码\2、Linux 移植驱动及应用\1、Linux 应用\实验代码\07. Linux 系统文件目录操作编程实验\实验代码\7.2”目录下的“myls.c”文件拷贝到该目录下。

执行命令：

```
$ gcc myls.c -o myls
$./myls example
```



实验代码如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <errno.h>
#include <unistd.h>
#include <string.h>
#include <time.h>
#include <pwd.h>
#include <grp.h>
#include <dirent.h>

#define N 128

int aflag = 0, lflag = 0;

void displayfile(char *name, char *showname);
void displaydir(const char *name);

int main(int argc, char *argv[])
```

```
{
    struct stat buf;
    int ch, i;

    if (argc < 2)
    {
        fprintf(stdout, "usage: %s filename\n", argv[0]);
        exit(-1);
    }

    while ((ch = getopt(argc, argv, "al")) != -1)
    {
        switch(ch)
        {
        case 'a':
            aflag = 1;
            break;
        case 'l':
            lflag = 1;
            break;
        default:
            printf("wrong option %c\n", optopt);
        }
    }

    //  printf("aflag=%d lflag=%d\n", aflag, lflag);

    for (i = optind; i < argc; i++)
    {
        if (stat(argv[i], &buf) == -1)
        {
            perror("stat");
            exit(-1);
        }

        if (!S_ISDIR(buf.st_mode))
            displayfile(argv[i], argv[i]);
        else
            displaydir(argv[i]);
    }
    if (lflag == 0)
        printf("\n");
    return 0;
}
```

```
void displayfile(char *name, char *showname)
{
	struct stat buf;
	struct tm *p;
	int i = 8;

	if (lflag == 0)
	{
		printf("%s ", showname);
		return;
	}

	if (stat(name, &buf) == -1)
	{
		perror("stat");
		exit(-1);
	}

	switch (buf.st_mode & S_IFMT)
	{
	case S_IFSOCK:
		printf("s");
		break;
	case S_IFLNK:
		printf("l");
		break;
	case S_IFREG:
		printf("-");
		break;
	case S_IFBLK:
		printf("b");
		break;
	case S_IFDIR:
		printf("d");
		break;
	case S_IFCHR:
		printf("c");
		break;
	case S_IFIFO:
		printf("p");
		break;
	default:
		printf("?");
```

```
    }
  while (i >= 0)
    {
        if ((buf.st_mode) >> i & 1)
        {
            switch (i % 3)
            {
            case 2:
                printf("r");
                break;
            case 1:
                printf("w");
                break;
            case 0:
                printf("x");
                break;
            }
        }
        else
        {
            printf("-");
        }
        i--;
    }

    printf(" ");

    /* hard links */
    printf("%d", buf.st_nlink);

    /* uid gid */
    printf(" %s %s    ", getpwuid(buf.st_uid)->pw_name, getgrgid(buf.st_gid)->gr_name);

    printf("%ld    ", buf.st_size);

    p = localtime(&buf.st_mtime);
    printf("%d-%d-%d %d:%d    ", p->tm_year + 1900, p->tm_mon + 1,
            p->tm_mday, p->tm_hour, p->tm_min);
    printf("%s\n", showname);
}
void displaydir(const char *name)
{
    DIR *dir;
    struct dirent *diritem;
```

```
    char str[N];

    dir = opendir(name);
    while ((diritem = readdir(dir)) != NULL)
    {
        if (strncmp(".", diritem->d_name, 1) == 0 && aflag == 0)
            continue;
        sprintf(str, "./%s", diritem->d_name);
        displayfile(str, diritem->d_name);
    }
}
```

五、实验数据（现象）处理分析

5.1 标准 I/O 实验

创建 8-io 文件夹，拷贝文件到 8-io 中，进入文件夹：

```
linux@ubuntu64-vm:~$ cd workdir/linux/application/
linux@ubuntu64-vm:~/workdir/linux/application$ sudo mkdir 8-io
[sudo] password for linux:
linux@ubuntu64-vm:~/workdir/linux/application$ cp /mnt/hgfs/share/实验代码/06.\
Linux系统标准IO实验/实验代码/6
61.c  64/
linux@ubuntu64-vm:~/workdir/linux/application$ sudo cp /mnt/hgfs/share/实验代码/
06.\ Linux系统标准IO实验/实验代码/61.c 8-io/ -a
linux@ubuntu64-vm:~/workdir/linux/application$ cd 8-io
```

运行对应程序，可以看到终端屏幕共输出了 1024 字节大小的内容，多余的内容没有输出，这就验证了全缓存的特点：当填满 I/O 缓存后才进行实际的 I/O 操作。

```
linux@ubuntu64-vm:~/workdir/linux/application$ cp /mnt/hgfs/share/实验代码1/06.\
 Linux系统标准IO实验/实验代码/* 8-io/ -a
linux@ubuntu64-vm:~/workdir/linux/application$ ls
5-shell  6-gcc  8-io
linux@ubuntu64-vm:~/workdir/linux/application$ cd 8-io
linux@ubuntu64-vm:~/workdir/linux/application/8-io$ ls
61.c  64  64.c
linux@ubuntu64-vm:~/workdir/linux/application/8-io$ gcc 61.c -o io_test
linux@ubuntu64-vm:~/workdir/linux/application/8-io$ ./io_test
000001002003004005006007008009010011012013014015016017018019020021022023024025026
027028029030031032033034035036037038039040041042043044045046047048049050051052053
054055056057058059060061062063064065066067068069070071072073074075076077078079080
081082083084085086087088089090091092093094095096097098099100101102103104105106107
108109110111112113114115116117118119120121122123124125126127128129130131132133134
135136137138139140141142143144145146147148149150151152153154155156157158159160161
162163164165166167168169170171172173174175176177178179180181182183184185186187188
189190191192193194195196197198199200201202203204205206207208209210211212213214215
216217218219220221222223224225226227228229230231232233234235236237238239240241242
243244245246247248249250251252253254255256257258259260261262263264265266267268269
270271272273274275276277278279280281282283284285286287288289290291292293294295296
297298299300301302303304305306307308309310311312313314315316317318319320321322323
324325326327328329330331332333334335336337338339340341342343344345346347348349350
```

5.2 Linux 系统文件目录操作编程实验

到 Linux 系统文件目录操作编程实验里面，我们可以看到执行下面命令后，得到了对应的输出结果。如下图所示：

```
$ gcc myls.c -o myls
$./myls example
```

```
linux@ubuntu64-vm:~/workdir/linux/application/8-io/dir$ sudo gcc myls.c -o myls
myls.c: 在函数‘displayfile’中:
myls.c:114: 警告： 格式‘%d’需要类型‘int’，但实参 2 的类型为‘__nlink_t’
linux@ubuntu64-vm:~/workdir/linux/application/8-io/dir$ ./myls example
2.c 1.c
linux@ubuntu64-vm:~/workdir/linux/application/8-io/dir$
```

运行命令后，与输出结果与预期的一样，并对内容进行读取识别，输出结果。

在第三个小实验和第二个小实验同理，学习用 Linux C 操作文件目录的方法。

六、实验结论：

通过本次实验，我深入理解了操作系统中系统调用和 API 的基本工作原理。系统调用为应用程序提供了对硬件资源的访问权限，而 API 则为开发者提供了更高层次的接口，简化了编程过程。实验过程中，我通过具体的实验操作，感受到了操作系统如何通过这些机制高效管理和调度系统资源，保证了程序与硬件之间的无缝连接。这样一来，程序员可以专注于功能的实现，而无需过多关注底层的硬件交互，极大地提高了开发效率。

另外，虚拟文件系统（VFS）的实验让我对文件系统管理有了更加深刻的认识。VFS 作为操作系统与具体文件系统之间的接口，提供了统一的访问方式，从而让操作系统能够在多个不同的文件系统之间切换而不影响程序的运行。通过实验，我理解了这一机制的灵活性和扩展性，它使得操作系统能够兼容多种类型的文件系统，同时也为开发者提供了更为简便的编程接口。

在文件操作的实验中，我特别关注了文件描述符和标准 I/O 的对比。标准 I/O 通过缓冲区管理提高了数据处理的效率，尤其是在处理大量数据时，缓冲区能够显著加速读写速度。而文件描述符则提供了更底层的控制，更加适合网络编程等需要高灵活性的场景。两者的差异让我认识到在实际开发中如何根据应用的需求来选择合适的 I/O 方式，优化程序的性能。

最后，实验让我意识到 I/O 缓冲机制的重要性。在处理大量数据时，合理的缓冲区管理能够显著提升程序的效率，但如果未能及时刷新缓冲区，可能会导致数据丢失或延迟输出。通过实验，我学习了如何在程序中合理控制缓冲区刷新时机，以确保数据的完整性和实时性。因此，掌握文件操作的每个环节，尤其是缓冲区管理，对于开发高效、稳定的程序至关重要。

七、指导教师批阅意见：

成绩评定：

指导教师签字：

年　　月　　日

八、备注：

注：报告内的项目或内容设置，根据实际情况可加以调整和补充，写不下可另附一页。